

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/376780127>

Bridging Workflow Automation Tools and EMF Modeling Ecosystems

Conference Paper · October 2023

DOI: 10.1109/MODELS-C59198.2023.00140

CITATIONS

2

READS

270

6 authors, including:



Francesco Basciani

University of L'Aquila

24 PUBLICATIONS 247 CITATIONS

[SEE PROFILE](#)



Ludovico Iovino

Gran Sasso Science Institute

114 PUBLICATIONS 1,501 CITATIONS

[SEE PROFILE](#)



Javier Luis Canovas Izquierdo

Universitat Oberta de Catalunya

96 PUBLICATIONS 1,612 CITATIONS

[SEE PROFILE](#)



Jordi Cabot

Luxembourg Institute of Science and Technology (LIST)

433 PUBLICATIONS 9,795 CITATIONS

[SEE PROFILE](#)

Bridging Workflow Automation Tools and EMF Modeling Ecosystems

Adiel Tuyishime,
Francesco Basciani,
Ludovico Iovino

*Gran Sasso Science Institute
L'Aquila, Italy*

Javier Luis Cánovas Izquierdo
*IN3 – UOC
Barcelona, Spain*

Jordi Cabot
*Luxembourg Institute of Science and Technology
Esch-sur-Alzette, Luxembourg*

Alfonso Pierantonio
University of L'Aquila, Italy

Abstract—A workflow specification is a detailed description of the steps, activities, and tasks that need to be completed in an automated way to achieve a specific goal. The advent of cloud-based infrastructure has enabled new possibilities, including adopting low-code approaches for service integration. Indeed, low-code development platforms (LCDPs) have recently gained prominence as they permit unskilled people to produce applications with little or no coding. Among LCDPs, integration platforms, such as Zapier, IFTTT, and Make.com, emerged for their cost-effectiveness and to reduce the tediousness of implementing service integrations. In such a context, existing platforms for building workflow automation (i.e., Workflow Automation Tools, WATs) offer pre-built integrations or create connectors to any service with REST APIs for data manipulation. WATs are primarily visual and offer no-code or low-code functionalities to build services. Model-Driven Engineering (MDE), on the other hand, is often compared to the low-code movement, and recent studies highlighted the substantial potential for cross-pollination. We experimented with a motivating example of how a WAT, called N8N, and pure MDE ecosystems built in the context of EMF Epsilon can share similarities and offer food for thought in implementing the same workflow scenario. Both approaches maintain pros and cons and show similarities and opportunities. For this reason, as a future direction, we propose a bridging mechanism between WATs and EMF ecosystems.

Index Terms—Workflow Automation Tools, Bridging, Modeling ecosystems, Low-code

I. INTRODUCTION

A workflow specification is a detailed description of the steps, activities, and tasks that need to be completed in an automated way to achieve a specific goal. Recently, no-code Workflow Automation Tools (WATs), such as Zapier¹, IFTTT², NodeRed³, N8N⁴, and Integromat/Make.com⁵ emerged for their cost-effectiveness by providing pre-built integrations and the possibility to create connectors to any service with REST APIs for data manipulation. They represent a successful attempt to adopt low-code development platforms (LCDPs) to create and manage workflows and integration processes in a

cost-effective and streamlined way. LCDPs allow unskilled users to develop applications and workflows, even visually, without writing complex and extensive code [1].

Model-Driven Engineering [2] (MDE) is often compared to the low-code movement since both aspire to improve and further automate the software development process by raising the level of abstraction and leveraging automation [1]. However, model-driven technologies target more language designers and offer them plenty of expressiveness at the risk of not being usable for non-technical people. Di Ruscio et al. [1] highlighted that being the two disciplines conceptually close, there is substantial potential for cross-pollination.

This paper explores data manipulation in WATs and discusses it compared with model management operations in MDE. We consider for this discussion two of the most used ecosystems in both disciplines, i.e., Eclipse Epsilon [3], as a framework for implementing pure MDE ecosystems; and N8N as one of the prominent WATs and the related ecosystem of services. This comparison outlines that they target different audiences and offer different levels of automation, complexity, and flexibility, and keeping the benefits of both approaches would be ideal. For this reason, we explore the bridge between WATs and MDE ecosystems.

II. RUNNING EXAMPLE

This section describes a motivating example that will be then implemented using two different solutions, one based on MDE tools and another based on WATs, to then compare and discuss how we can combine both solutions⁶.

Scenario. The departments of a university have international collaborations with professors and researchers who can be invited to visit hosting research groups (e.g., for giving a course or for invited visiting positions). A university faculty member can open an “invited guest position” on the system, where he/she must express information related to the guest position, including the period of stay, title of collaboration, hours of lectures, and course title. The remuneration can be calculated according to the department’s rules and period of

¹<https://zapier.com>

²<https://ifttt.com>

³<https://nodered.org/>

⁴<https://n8n.io/>

⁵<https://www.make.com>

⁶A detailed description of the scenario is provided in [4].

stay and collaboration. After this information has been set up, an invitation letter is generated and eventually sent to the invitee to specify the financial arrangement and all the information that can be useful and used by the invitee to motivate their visit. As long as we specify the course title, the dates, and an abstract, a web page can be generated and published online to announce the course or the novel collaboration. Concerning the payment, when a new guest is invited, a new payment must be emitted, and this should be managed by another system when the collaboration has been completed and confirmed.

III. SOLVING THE EXAMPLE: THE MDE VERSION

In the following, we present the implemented model management operations to support the described scenario, implemented with a modeling ecosystem. In MDE, a modeling ecosystem [5] is composed of a large number of artifacts pursuing a common scope that are usually defined starting from a metamodel, which represents its nucleus [5]. A model management framework defines and implements model management operations, such as transformation or code generation, with the main aim of manipulating modeling artifacts [6] to pursue a common goal. Even if a *minimal* MDE infrastructure can be defined as a set of models and a transformation built on top of their metamodels pursuing a pre-defined scope, modeling ecosystems are usually more complex and articulated. Tools and languages for manipulating models are usually chained or composed to implement the modelers' goals, creating links between modeling elements belonging to different models [7].

In our case study, we use Eclipse Modeling Framework (EMF) [8] and Epsilon [3] for the modeling environment, and the ETL language to implement the model-to-model transformation, as well as the EGL language for model-to-code, and model-to-text transformations. In addition, model validation and the interaction with external services are done with EVL and EOL, respectively. Finally, ANT is used for concatenating the various operations. In *GuestInvitation* and *PersonRemuneration* metamodels (cf. Figures 1 and 2 in [4]), all the concepts for the systems managing the guest invitation and payments are formalized, as described before. To fully support the case study specification, we would need to implement (1) a model transformation from the *Guest Invitation* to the *PersonRemuneration* metamodel, (2) a code generator generating web pages for the course/collaboration, (3) a document generator for generating the invitation letter, (4) a script invoking an external Java service for sending emails, and finally (5) all the operations must be supported by validation scripts, assuring that the model instances are correct.

Model Transformation. Model transformations are usually defined with languages and tools that transform one model into another. In EMF [9], ATL (Atlas Transformation Language) [10], QVT (Query/View/Transformation) [11], VIATRA (M2M) [12], Epsilon Transformation Language (ETL), Epsilon Object Language (EOL) are some of the tools that can be used to implement a transformation. ATL, ETL, and

VIATRA are rule-based model transformation languages in which transformational mappings are expressed via rules, where the modeler can set source and target metamodel elements. These transformation languages express rules with text-based syntaxes, whereas others use graphical editors to offer an intuitive way of representing rules. In Henshin [13], for instance, objects are referred to as nodes, and links between objects are as edges. To support the required scenario, an ETL model-to-model transformation can be developed to generate an instance of the *PersonRemuneration* metamodel from the *GuestInvitation* metamodel.

In a nutshell, each instance of *Guest* in the input model is transformed into a new instance of *Person* in the output model, with the same information, e.g., *firstName* and *lastName* based on the structure of the target metamodel. Also, a new *Remuneration* instance will be generated, including the overall amount of remuneration the guest is receiving.

Model Validation. Data validation within an execution flow provides benefits such as timely error detection, enhanced result reliability, prevention of costly errors, improved performance, data security, and enhanced user experience. When it comes to the execution flow, incorporating data validation can bring numerous advantages. For instance, it can help detect errors in a timely manner, thus helping prevent costly mistakes down the line. Also, it can help ensure the reliability of results, which is critical for many applications or to prevent unexpected behavior. By validating data, the system can ensure that the data is clean and well-structured. Furthermore, incorporating data validation can enhance the user experience, as it may help ensure that it is intuitive and user-friendly. Data validity checks can be performed in the EMF environment using the Epsilon Validation Language (EVL). In our case study, for instance, two validation rules can be defined in the context of the *Remuneration* metaclass to verify (1) if the total amount to be remunerated is positive and (2) if the application status is in the *ReadyToBeSent* state. This prevents remunerating from being computed from negative amounts or when the process is incomplete.

Code generation. Template-Based Code Generation (TBCG) is a code synthesis technique generating code from templates, which are high-level specifications mixing static parts and dynamic parts filled with variables updated from models [14]. In the MDE ecosystems, some TBCG examples are Aceleo, EGL, or XPand. In an EMF environment, we use Epsilon's EGL transformation language, which the EGX coordination language invokes. Thus an EGX script with a rule for each instance of the *Guest* metaclass in the input model creates an HTML file using the corresponding EGL template where HTML code is interleaved with model navigation constructs. The EGL template generates HTML code by means of the placeholders that fetch data directly from *Guest* instances.

Document generation. Document generation is an automation task related to transformations and code generation, as they can generate any string or format. Indeed, code generation is also called *Model to Text* transformations. In the MDE ecosystems,

we can implement document generation in multiple ways and using multiple languages and tools. Usually, EMF Aceleo or EGL can generate documents precisely as we have developed the source code before. In document generation, a set of rules defined in an EGX file create the invitation letter for the guests. These rules generate a document using the *Guest* instances, and the target language can be a markup language with the extension “.md” or even a Word document. As for the previous task, the EGL template file has a structure that generates text with placeholders allowing the navigation to the input model. This document can then be used in our case study for attaching the invitation letter to the email addressed to the *Guest* that can be automatically sent.

Interaction with external services. To automate the sending of emails to the guest, the Epsilon Object Language (EOL) allows the creation of objects of the underlying programming environment using native types. For instance, an EOL script can use an external service to send an email via Java (e.g., Google API or SMTP library in Java). A Java class could provide a basic configuration for the SMTP host, port, and other required data to send emails programmatically.

Workflow automation in Epsilon. When all these artifacts have been developed, model management activities must be combined together to form the entire workflow. The Epsilon framework provides a set of Apache ANT tasks for assembling multi-step automated build processes. The ANT script reproduces the previous steps, that is, it first loads the input model, and then the target model to be provided as output in the model-to-model transformation is also set. The workflow, therefore, involves validating the input model that may trigger, if positively evaluated, the model-to-model and model-to-code transformations. The invitation email is generated, and finally, through EOL, the external Java service that sends the emails with the input model information is called.

IV. SOLVING THE EXAMPLE: THE WAT VERSION

Workflow automation tools play a significant role in the low-code landscape by offering a visual and declarative approach to designing and orchestrating business processes and automation workflows. Within the realm of low-code development, these tools serve as a fundamental framework to define the flow, logic, integration, and dependencies of tasks and activities within software applications. Workflow tools are useful for automating repetitive tasks, streamlining complex business processes, and enabling efficient collaboration between users and systems. Through their visual representation of workflow logic, these tools enable users to define and connect various steps, actions, and decision points, resulting in the automated and standardized execution of tasks. This automation streamlines operations, reduces manual effort, eliminates errors, and enhances productivity by handling routine activities like data transformation, validation, notifications, and approvals. Moreover, workflow tools often come with a wide range of pre-built components and connectors, simplifying the seamless integration of external systems, databases, APIs, and services. This integration capability empowers users to rapidly build

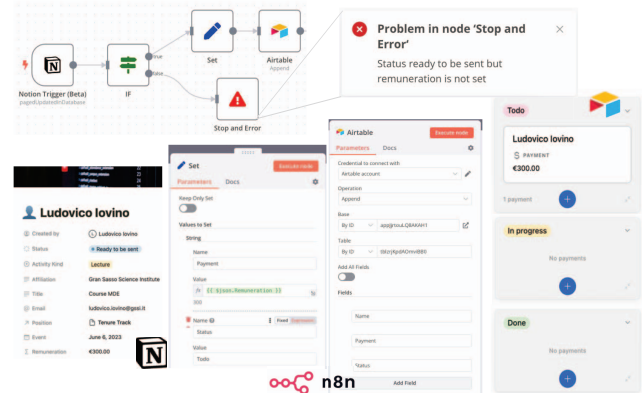


Fig. 1. Model Transformations in N8N.

end-to-end solutions by leveraging existing functionality and resources. This paper uses N8N as WAT because of its main advantage compared to the other platforms; it can be self-hosted as a deployment option.

Nodes are the key building blocks of a workflow in N8N, and they can be used to perform a range of actions, including starting the workflow, fetching or sending data to the next node, and manipulating data. In this section we show how to implement the same scenario reported in sec. II with N8N. In Fig. 1, a node mapping data from a Notion⁷ to an Airtable⁸ record is defined. Both tools are Low-Code Platforms (LCPDs) for building data-driven applications, offering an easy way to configure data schema. They offer multiple views but share a tabular view in which first the user can define the entities, and then the table can be filled with records. In these two tools, the data structure corresponds to our metamodels, *Guest Invitation* and *Person Remuneration*, respectively.

In the screenshot in Fig. 1, the node workflow configuration allows us to define the mapping for transforming data from the Notion entities, i.e., guest invitations, to the Airtable ones, representing the remuneration data structure. In particular, this node configuration can be considered a transformation from guests to a flat list of persons that can be used to manage and keep track of remuneration. With the Notion tool, the guest invitations are defined, and with Airtable, the remuneration records are managed, and this node, shown in the center of the picture, automatically flows data.

In this example, we determined how N8N implements transformations by establishing a visual mapping between nodes. Rule-based model transformations have a straightforward way of defining model feature mapping; in rule bindings, the model slots are filled by expressing the mapping directly. Instead, in N8N, syntactic aspects are demanded by the node receiving data. This means that every node can have a different setting and influence the definition of the mapping. For instance, the Airtable node expects to receive data with fields corresponding to the same data structure. Indeed, the guest's name will

⁷www.notion.so

⁸www.airtable.com

be automatically propagated. In contrast, the payment must be filled with the value of the *remuneration* and needs an additional trick to propagate values. The action is performed by using a node SET, which is used to set custom variables, and then by setting a variable named *Payment*, the Airtable node will be able to set correctly the value of the field labeled with the same name. This workaround is effective but requires an additional node to implement what could be obtained with a single M2M transformation. The target visualization based on a Kanban view can be easily realized with an additional code generator transforming the output of the transformation into the required view.

As seen in the model validation, in N8N, a node that validates data to be prepared to start the remuneration activity can be created. This validation node checks if there is anything to refund by verifying if the total amount of money to remunerate is greater than zero and if the request status is *ReadyToBeSent*. This is easily done using a conditional node, i.e., IF checking if the total amount of remuneration is greater than 0 and the record status is *ready to be sent*. If this check fails, an error will appear, as shown in Fig 1 (error popup).

This is again a very similar result to the one that can be obtained with EVL rules, but in this case, the modeler can express all the constraints in a single script with a very powerful syntax allowing the specification of constructs that can reach a high level of complexity. N8N and, in general, WATs offers node for conditional constructs that can be chained with the same result but with a higher level of effort to express complex validation constraints.

Concerning the code generation for getting the web view showing the required information retrieved from the models, a specific node in N8N permits the generation of HTML. This node, from the given Notion database, generates an HTML page containing information about the scientific collaboration. The template interleaves HTML (target platform language) with the JavaScript code used to access data propagated from the previous nodes. The generated target can be exactly the same as a web page generated with EGL. This is a case limit for workflow and N8N since WATs are often cloud-based (and web-based), and then it would not be possible to target other languages for code generation than HTML, which can be rendered internally by the browser. Indeed, in N8N, we could not find any other nodes for code generation⁹.

The last part of the scenario needed to complete the example is document generation and email sending to the guest. This is supported by a GoogleDoc node, where the user can use *search and replace* to fill values in an online Google Doc. To the best of our knowledge¹⁰, this is the only way to implement document generation in this setting. We used a “template” version of GoogleDoc, where we manually specified placeholders with a pre-defined syntax, i.e., {{placeholder}}, that will be replaced at runtime with values retrieved from the data flow when the workflow is executed. Even though it is effective, the

⁹Custom nodes could be developed, but this is out of scope.

¹⁰The same method can be used with Microsoft SharePoint documents



Fig. 2. Code generation in N8N (HTML).

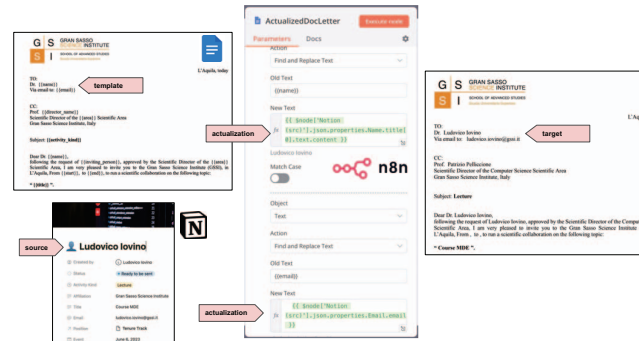


Fig. 3. Document generation in N8N.

main drawback of this method is that the user has to specify an object mapping for each placeholder.

This is quite demanding concerning a code generator engine such as EGL or Acceleo. Then the workflow goes on with a PDF generation node from the GoogleDoc and a Gmail node sending to send the email to the guest with the attached invitation letter.

V. BRIDGING WATs AND MDE ECOSYSTEMS

We can present some findings and comparisons based on this small-scale experiment. Both development methodologies can help solve the motivating example but with some divergences, similarities, and opportunities.

Workflow propagates data through the nodes, which can be selectively used at all stages. If we consider a workflow as a model transformation chain, this will enable multiple input sources. Instead, rule-based transformation languages can be limited to the number of source/target models in the direct transformation. For instance, ETL can have transformation rules that consume elements from different source models but cannot have multiple source parameters as source elements of the rule. It represents a substantial similarity since, in N8N, the user can select the node from which to obtain data to propagate and transform to the next node. This is similar to what can be done in ATL when the modeler declares the input model and uses OCL [15] to navigate it in the transformation. **Specific syntax for M2M.** In EMF, M2M can be expressed with visual or textual syntax, depending on the transformation

language, whereas in WATs, mappings are always specified with predefined forms and node concatenation. Besides, in EMF, models are queried, for example, via OCL or EOL, whereas N8N uses Javascript (or proprietary micro syntax in other WATs).

Limited code generation in WAT. Code generation is divided into two subgroups: generating code and generating HTML. Generating code does not seem possible with N8N. However, HTML code generator can be very easy to use, and the node provided by N8N is very similar to an EGX or an Aceleo template.

Similar document generation capabilities. Document generation is possible either way, even though with N8N, we have found a workaround based on searching and replacing strings in manually created templates that over-complicate the specification. Generating documents in EMF requires writing a template that involves model navigation expressions able to retrieve the relevant information to be injected in the targeted system. Searching and replacing in a document requires the specification of multiple fields in the N8N form for each value in which we need a placeholder accessing data. In template-based code generators like Aceleo or EGL, placeholders are specified textually, with less effort when placeholders are many.

Domain-specific syntaxes and editors in MDE. Modeling ecosystems can include multiple artifacts devoted to expressing the models in a domain-specific way. For instance, models can be expressed with DSLs that can be graphical or textual, depending on the nature of the application domain [16] with great flexibility. WATs are limited to pre-defined custom nodes bridging existing tools in this case, but the composition of services is assured by nature, whereas in EMF, we had to use another tool. Most of the data-driven specification tools that can be used for data management offer a tabular or pre-defined view that cannot be extended much (e.g., Notion or Airtable). On the other side, modeling editors can be very advanced and customized. For instance, editors offering custom palettes and validation constraints, or even form-based editors [17], can easily be developed in EMF. The main takeaway message is that integrating services in WATs can be effective and somewhat effortless, but some limitations can hinder their full potential. At the same time, MDE ecosystems can offer custom modeling artifacts development that can fit some domain requirements not implementable in WATs. Given these premises, we can suppose that getting the best of both worlds can be of great benefit. To this end, we propose a bridging mechanism by developing approaches similar to EMF-REST [18]. In a nutshell, EMF-REST enables the generation of RESTful Web APIs out of EMF models, thus promoting model management in distributed environments. With this kind of solution, developers can leverage the full potential of Platform-as-a-Service (PaaS) providers and effectively exploit their scalability capabilities for model management. Most WATs offer integration by exploiting REST API to build custom nodes that can be used in the workflow specification.

Given that EMF-REST exposes the EMF instances using REST APIs, it seems interesting to build a custom node, that can be used within WATs to read and write EMF models. Integrating MDE modeling platforms with WATs presents a significant opportunity to enhance the effectiveness and efficiency of software development processes, particularly in light of the growing prominence of LCPDs. By leveraging the strengths of MDE and WATs, business companies, institutions, and practitioners can streamline and automate their business tasks, leading to improved productivity and quality outcomes. This mechanism does not force a user to migrate to a platform losing the advantages of one or the other and avoiding the exposition of the vendor lock-in problem.

REFERENCES

- [1] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, "Low-code development and model-driven engineering: Two sides of the same coin?" *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022.
- [2] D. C. Schmidt *et al.*, "Model-driven engineering," *Computer-IEEE Computer Society*, vol. 39, no. 2, p. 25, 2006.
- [3] "Eclipse Epsilon website," <https://eclipse.dev/epsilon/>, accessed: 2023-08-21.
- [4] L. Iovino, B. Francesco, and T. Adiel, "Model management operations in MDE," https://github.com/gssi/emf_workflow_project/tree/main, 2023.
- [5] L. Iovino, A. Pierantonio, and I. Malavolta, "On the Impact Significance of Metamodel Evolution in MDE," *J. Object Technol.*, vol. 11, no. 3, pp. 3–1, 2012.
- [6] R. F. Paige, D. S. Kolovos, L. M. Rose, N. Drivalos, and F. A. Polack, "The Design of a Conceptual Framework and Technical Infrastructure for Model Management Language Engineering," in *Int. Conf. on Engineering of Complex Computer Systems*, 2009, pp. 162–171.
- [7] A. Abouzahra, A. Sabraoui, and K. Afdel, "Model composition in Model Driven Engineering: A systematic literature review," *Information and Software Technology*, vol. 125, p. 106316, 2020.
- [8] D. Steinberg, F. Budinsky, E. Merks, and M. Paternostro, *EMF: Eclipse Modeling Framework*. Pearson Education, 2008.
- [9] F. Budinsky, D. Steinberg, E. Merks, R. Ellersick, and T. Grose, *Eclipse Modeling Framework*. Addison Wesley, 2003.
- [10] F. Jouault, F. Allilaire, J. Bézivin, I. Kurtev, and P. Valduriez, "ATL: a QVT-like transformation language," in *ACM SIGPLAN symposium on Object-oriented programming systems, languages, and applications*, 2006, pp. 719–720.
- [11] "MOF 2.0 Query / Views / Transformations," *OMG: RFP, OMG Document*, vol. ad/2002-04-10, 2002.
- [12] G. Csértán, G. Huszerl, I. Majzik, Z. Pap, A. Pataricza, and D. Varró, "VIATRA-visual automated transformations for formal verification and validation of UML models," in *Int. Conf. on Automated Software Engineering*. IEEE, 2002, pp. 267–270.
- [13] T. Arendt, E. Biermann, S. Jurack, C. Krause, and G. Taentzer, "Henshin: advanced concepts and tools for in-place EMF model transformations," in *Int. Conf. on Model Driven Engineering Languages and Systems*. Springer, 2010, pp. 121–135.
- [14] E. Syriani, L. Luhunu, and H. Sahraoui, "Systematic mapping study of template-based code generation," *Computer Languages, Systems Structures*, vol. 52, pp. 43–62, 2018.
- [15] M. Richters and M. Gogolla, "OCL: Syntax, semantics, and tools," in *Object Modeling with the OCL: The Rationale behind the Object Constraint Language*. Springer, 2002, pp. 42–68.
- [16] S. Meliá, C. Cachero, J. M. Hermida, and E. Aparicio, "Comparison of a textual versus a graphical notation for the maintainability of MDE domain models: an empirical pilot study," *Software Quality Journal*, vol. 24, pp. 709–735, 2016.
- [17] L. Bettini, "Developing user interfaces with EMF parsley," in *Int. Conf. on Software Paradigm Trends (ICSPT-PT)*. IEEE, 2014, pp. 58–66.
- [18] H. Ed-Douibi, J. Cánovas Izquierdo, A. Gómez, M. Tisi, and J. Cabot, "EMF-REST: generation of RESTful APIs from models," in *Symposium on Applied Computing*, 2016, pp. 1446–1453.